Image Processing Using GPU's For Analysis

Brook Hamilton

hamilbro@oregonstate.edu

CS 475: Parallel Programming

Project 6

Professor Jesse Rosenzweig

November 8, 2026

Oregon State University - Cascades

Introduction:

This project is derived from a recent study that demonstrated that coin flips are slightly biased towards the side that the coin started on. We are essentially re-creating this study, by automating the detection of heads and tails using image processing and analysis. This is done by processing black and white images of coins heads or tails, in order to make the analysis simpler, we are only using images of both sides of dimes where each image lines up perfectly with a slot on a 2 dimensional grid. By utilizing GPUs for parallel processing we are going to be able to quickly analyze images and count how many coins landed heads up.

The images we are processing are provided in PGM format; a byte map with each byte representing one greyscale from 0-255. This format makes the images very easy to understand and process. Using these images, we will be using the SAD calculation to tell the difference of every pixel in the comparison area and compare it with a template, a low SAD indicates a match as compared to this template

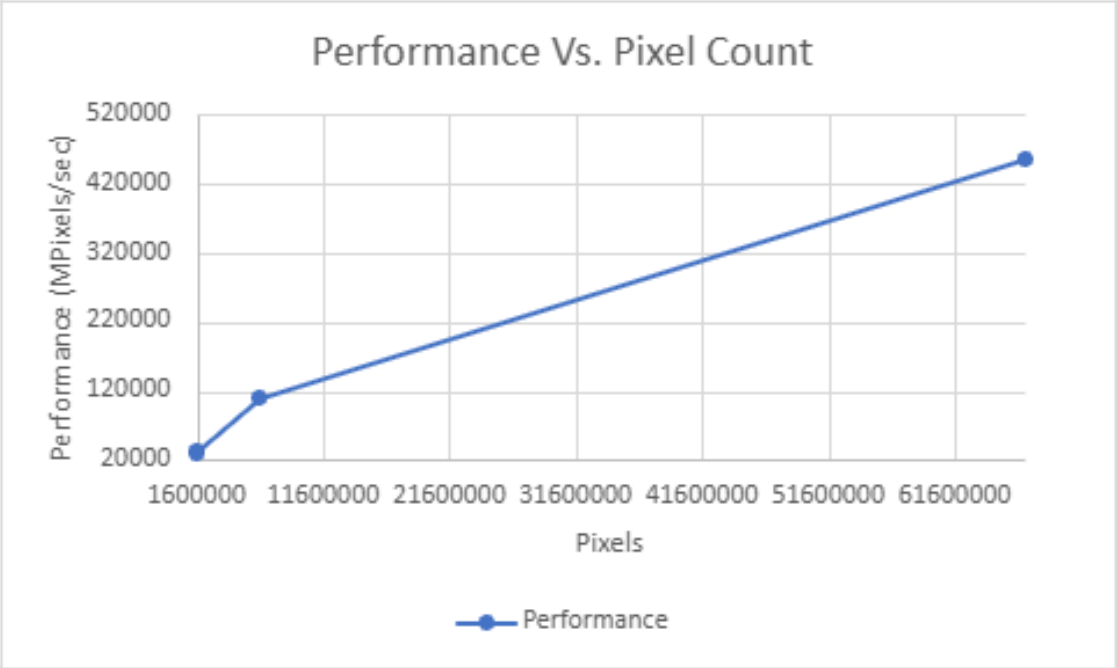
Machine Details:

Machine: Pelican Virtual Machine

Architecture:	X86_64
CPU(s):	32
Thread(s) per core:	2
Core(s) per socket:	8
Operating System:	Linux
Compiler(s):	gcc/7.5, NVIDIA nvcc CUDA 11.2
GPU:	NVIDIA GeForce GTX 1080 Ti

Results:

Pixels	Performance	Percent Heads
1638400	29711.31	100
1638400	31361.75	60
6553600	109299.53	58.5
67108864	455283.97	51.2



Discussion:

The count_coins_gpu() function is the main driver and purpose of this project, its purpose is to check the image provided against a coin template on the gpu and return the number of tiles (numCoins) that match. This is done twice, once with coin_head and once with coin_tail. The image passed to the function is treated as a grid of tiles, all the size of a coin. The GPU launches one block per tile each with THREAD_GRID_X * THREAD_GRID_Y or 16 * 16 threads.

Function Structure:

- Get grid size:

```
C/C++
unsigned int x_blocks = image_width/coin_width;
unsigned int y_blocks = image_height/coin_height;
unsigned int num_blocks = x_blocks*y_blocks;
```

- Copy memory to the GPU
    - Image is only copied the first time through

```C/C++
if (bFirstTime)
{
      cudaMemcpy( device_image, image, image_height*image_width,
cudaMemcpyHostToDevice);
      bFirstTime = false;
}
```

- Execute kernel

```C/C++
dim3 threadsPerBlock(THREAD_GRID_X, THREAD_GRID_Y);
dim3 numBlocks(x_blocks, y_blocks);
device_count_coins<<<numBlocks, threadsPerBlock>>>(...);
cudaDeviceSynchronize();
```

- Copy results back and add them up

```C/C++
cudaMemcpy(count_per_block, device_count_per_block, size_count,
cudaMemcpyDeviceToHost
);
...
for (int i = 0; i < num_blocks; i++)
{
      numCoins += count_per_block[i];
}
```

The device_count_coins() function is another large part of this function, each thread is responsible for finding its position in the tile and within the image as a whole. It then computes the difference using SAD (Sum of Absolute Differences). If the sum is below CORRELATION_THRESHOLD it is a match and is assigned a 1 for that block's id within device_count_per_block, if not it is assigned a zero.

Conclusion:

The performance scaling seen in the graph and table is very interesting to visualize but also makes a lot of sense when thinking about how the GPU works with parallelization. There is a very obvious increase in performance at a nearly linear level as the number of pixels increases.

More pixels means the image is larger which in turn means that there are more threads working at the same time and less overhead when launching the kernel within the GPU. the smaller images are also unable to fill the GPU, so the smaller images cannot keep all cores busy at the same time so it is just wasted multiprocessors. In order for this program to be improved for maximum efficiency it should always be processing images larger than or equal to 8192 X 8192 pixels in order to maximise the GPU multiprocessors.

The results of the processing and analysis from a functional perspective are seen in the percent heads column of the table, which displays the percentage of identified coins that ended up being heads. As seen the program was able to accurately distinguish between heads and tales as compared to the provided templates and count how many of each there are.